

@SpringBootApplication

@ComponentScan + @SpringBootConfiguration + @EnableAutoConfiguration

What is Component Scanning?

When a Spring Boot application starts, it creates a **Spring Container (ApplicationContext)**.

After creating the container, Spring must answer one critical question:

Which classes should be converted into beans?

Spring cannot create beans from every Java class in the project.

So it needs to know **where to look**.

That is the job of **Component Scanning**.

Component Scanning is the process by which Spring:

- Searches certain packages
- Finds classes with Spring annotations (@Component, @Service, @Repository, @Controller)
- Creates objects for them
- Stores them inside the IoC container

Where Does Spring Start Scanning?

In a Spring Boot application, scanning begins from the **main class**.

Example:

```
package com.project.ecom;

@SpringBootApplication
public class EcommerceApplication {
    public static void main(String[] args) {
        SpringApplication.run(EcommerceApplication.class, args);
    }
}
```

Spring will scan:

```
com.project.ecom  
com.project.ecom.controller  
com.project.ecom.service  
com.project.ecom.repository
```

But **not**:

```
com.project.payment
```

Because that package is **outside** the main class package.

Why Your Beans Are Sometimes Not Found

Imagine this structure:

```
com.project.ecom      ← main class here  
com.project.ecom.controller  
com.project.ecom.service  
  
com.project.ecomPayment ← separate package
```

Spring will scan:

```
com.project.ecom and sub-packages
```

But it will **ignore**:

```
com.project.ecomPayment
```

So beans inside ecomPayment will NOT be created.

Fixing This with @ComponentScan

Option 1 – Scan both explicitly

```
@SpringBootApplication
@ComponentScan(basePackages = {
    "com.project.ecom",
    "com.project.ecomPayment"
})
public class EcommerceApplication { }
```

Now Spring scans both packages.

Option 2 – Scan root package

```
@ComponentScan(basePackages = "com.project")
```

This scans **everything** inside com.project.

Using @Configuration for Component Scanning

Let's create Config class and annotate the below format:

```
@Configuration
@ComponentScan(basePackages = "com.project.ecomPayment")
public class PaymentConfig {
}
```

Without @Configuration, Spring ignores this class.

Why?

Because only @Configuration classes are loaded into the IoC container at startup.

Once this config is loaded, Spring includes that package in scanning.

What is @SpringBootApplication?

This is not a single annotation.

It is a **combination** of

@SpringBootConfiguration, @EnableAutoConfiguration, @ComponentScan

@SpringBootApplication =

@SpringBootConfiguration + @EnableAutoConfiguration + @ComponentScan

```
@SpringBootApplication =  
@SpringBootConfiguration  
@EnableAutoConfiguration  
@ComponentScan
```

@SpringBootConfiguration

Internally:

@SpringBootConfiguration = @Configuration

It allows:

- Bean definitions
- Component scanning
- App configuration

It marks your main class as a **Spring configuration class**.

@EnableAutoConfiguration (Very Important)

This is what makes Spring Boot “**Boot**”.

Spring looks at:

- Your classpath
- Your dependencies
- Your environment

Then it auto-configures things.

Example:

If you add:

spring-boot-starter-web

Spring Boot automatically configures:

- DispatcherServlet
- Tomcat
- JSON converters
- Request mappings

You didn't write a single line — `@EnableAutoConfiguration` did it.

How All This Works Together

When your app starts:

1. `@SpringBootApplication` is found
2. `@SpringBootConfiguration` creates IoC container
3. `@EnableAutoConfiguration` loads required infrastructure
4. `@ComponentScan` finds your beans
5. Beans are created and wired
6. Application is ready

Interview-Grade Questions

Q1. What does @ComponentScan do?

Tells Spring which packages to scan for beans.

Q2. Why is my bean not getting created?

It is outside the component scan path.

Q3. What is @SpringBootApplication internally?

A combination of Configuration, AutoConfiguration, and ComponentScan.

@SpringBootApplication =

@SpringBootConfiguration + @EnableAutoConfiguration + @ComponentScan

Q4. What does @EnableAutoConfiguration do?

Automatically configures Spring based on dependencies.

One-Line Summary

“Spring Boot starts by creating a container, auto-configuring infrastructure, and scanning packages for beans — all driven by @SpringBootApplication.”